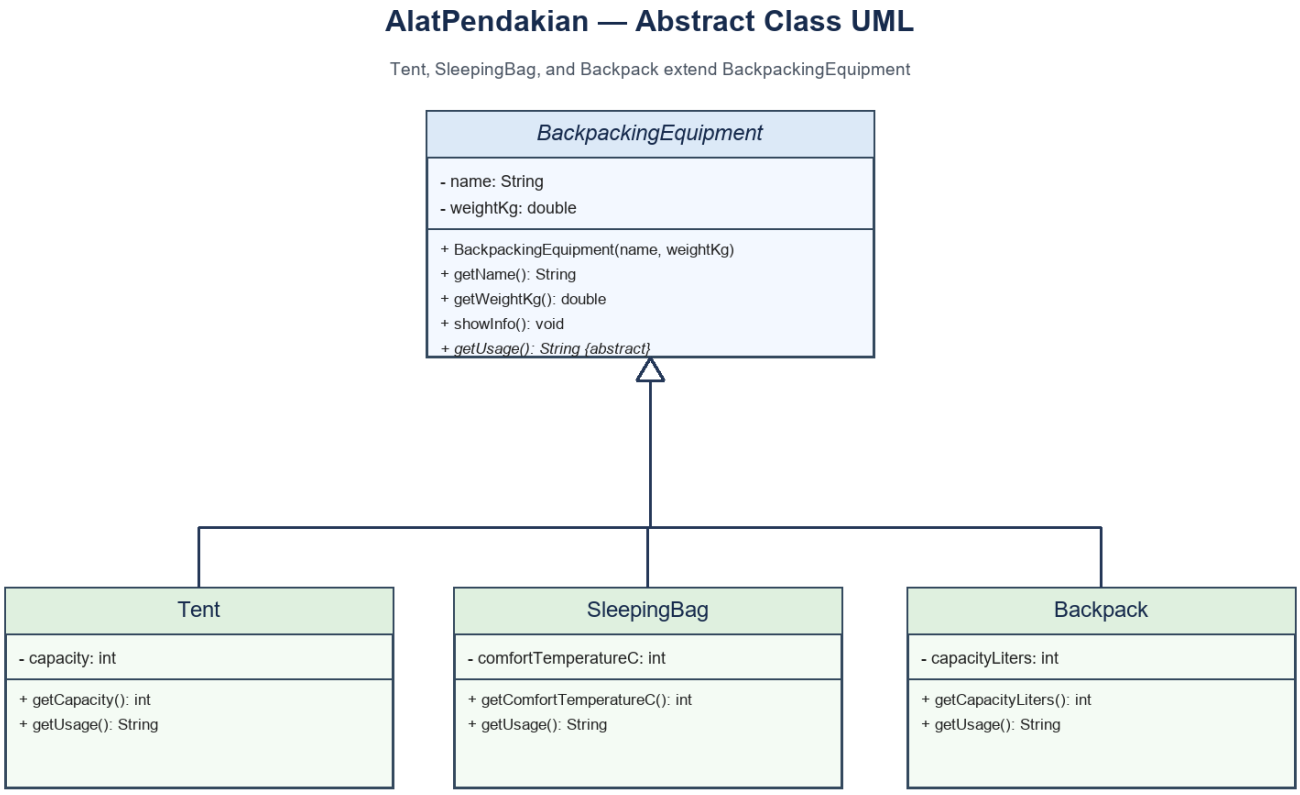
ABSTRACT CLASS — HIKING EQUIPMENT

Name: Nabeel Haqqi Elraf | Student ID: 254107060086 | Class: SIB 2G

Case Study: AlatPendakian (Hiking Equipment)

This case study models hiking equipment with an abstract superclass and concrete subclasses, following the Meet 9 theory material.

UML Class Diagram



The italicized superclass and {abstract} operation indicate abstraction. Hollow triangles point to the superclass.

Abstract Class and Method

BackpackingEquipment is the abstract class. It contains shared state (name and weightKg), a constructor, getters, and the concrete **showInfo()** method. It cannot be instantiated directly because it represents the general concept of equipment, not one specific item.

getUsage() is abstract: it specifies what every item must describe but does not define how. Tent, SleepingBag, and Backpack override it with type-specific descriptions.

Why Use Abstraction?

The subclasses share their name, weight, and information display, so these belong in BackpackingEquipment. Their usage differs, making one generic implementation unsuitable. Abstract getUsage() requires every concrete subclass to provide its own behavior; showInfo() can then reuse it polymorphically.

Classes and Responsibilities

• BackpackingEquipment — abstract parent; shared data; concrete showInfo(); abstract getUsage().

• Tent — concrete subtype with capacity and shelter usage.

• SleepingBag — concrete subtype with comfortTemperatureC and warmth usage.

• Backpack — concrete subtype with capacityLiters and carrying usage.

Implementation and Conclusion

The Java source files are provided in MEET_9/TEORI. Main creates the concrete subclasses and calls showInfo(); the abstract class is not instantiated. The superclass centralizes common features, while getUsage() guides each subtype to describe its own purpose.